Context-Aware Music Companion Using Wearable Health Data

Choosing the right song, from your own Spotify library, for the moment you are in

ROYER, Clément
Dept. Computer Science
Hanyang University
Seoul, S. Korea
clement.royer@edu.devinci.fr

VANDEN ABEELE, Cyprien
Dept. Computer Science
Hanyang University
Seoul, S. Korea
cyprien.vanden_abeele@edu.devinci.fr

MISKULIN, Lucas
Dept. Computer Science
Hanyang University
Seoul, S. Korea
lucas.miskulin@edu.ece.fr

MEKIDECHE, Riad
Dept. Computer Science
Hanyang University
Seoul, S. Korea
riad.mekideche@edu.devinci.fr

Group G02 "FireStarter" — Repository: https://github.com/SkaosDev/Harmony

***Abstract*—Waking up, stress, an afternoon energy dip and falling asleep are moments in which the right music can help, but nobody wants to search for it at that moment. We propose an iPhone app that picks songs from the user's own Spotify library to fit the moment. Each saved song gets two simple scores (how calm or energetic it is, and how positive it sounds) from the public MuSe dataset or, when missing, from a language model. From the Apple Watch heart rate, steps and sleep, and the time of day, a small model estimates whether the person is more or less activated than usual. Simple rules then play a short queue that starts close to the person's current state and moves step by step toward a calmer or more energetic state. The model is trained on EmoPhone, a dataset of 297 students in Korea who wore a watch and reported their mood several times a day; first tests give modest but real results, which we report honestly. The tech stack is as follows: Swift and SwiftUI for the app, HealthKit for watch data, the Spotify Web API and iOS SDK for the library and playback, Python and scikit-learn for training, and Core ML to run the model on the phone, so that health data never leave it.**

***Index Terms*—music recommendation, wearable data, context-aware app, Spotify, iOS, Core ML**

TABLE I
ROLE ASSIGNMENTS

Role	Name	Task description
User	Cyprien	Uses the app every day with a Spotify account and a smartwatch; notes whether the songs fit the moment; reports bugs; runs the pilot test.
Customer	Cyprien	Checks that each feature answers the needs of Section II-B; approves mock-ups and demos; decides trade-offs such as privacy versus accuracy.
Software developer	Riad	Builds the iPhone app: Spotify sign-in and library import, reading of watch data, screens and playback.
Software developer	Lucas	Builds the data and AI part: song scores, state model, song-choice rules and evaluation scripts.
Software developer	Clément	Works on both the iPhone app (Swift) and the AI part; reviews code; maintains the GitHub repository and the documents; holds the Apple and Spotify developer accounts.

I. ROLE ASSIGNMENTS

Table I gives the roles. The group has no leader: three members develop, every pull request is reviewed by another member, and decisions are taken together at a weekly meeting.

II. INTRODUCTION

A. Motivation

Poor sleep and stress are an everyday problem in Korea. A report based on a KB Financial Group study gives an average sleep time of 471 minutes per day, 30 minutes below the OECD average, and cites a Philips survey in which only 41% of Koreans were satisfied with their sleep, with stress as the first cause of disturbed sleep [1].

Music can help: a Cochrane review of 13 randomised trials (1,007 adults) found that listening to music improved sleep quality by 2.79 points on the Pittsburgh Sleep Quality Index, with moderate certainty of evidence [2]. This is evidence about music in general; it does not show that our app will improve well-being, and our pilot will not test that.

Music streaming is part of daily life, yet streaming platforms still ignore the body of the listener. Our first users are students and young workers who listen to music on their phone, wear a smartwatch and have irregular schedules. They want the music they already like, at the right moment, with one tap. The team members belong to this group and act as users during development.

B. Problem Statement (User Needs)

Given the time of day and the recent signals of the person's body compared with their own usual values, which songs from their own library should be played, and in which order, to help them through the moment? Table II lists the four situations we address.

TABLE II
TARGET SITUATIONS

Situation	Typical moment	Goal of the music
Wake-up	Morning, after sleep	Gently more energetic
Stress	Any time of day	Calmer
Low energy	Afternoon slump	More energetic
Wind-down	Evening, night	Calm and sleepy

- **N1. No effort:** a tired or stressed person does not want to search for a playlist; one tap should start suitable music.
- **N2. My own music:** songs come mainly from the user's library, never from generated audio.
- **N3. Fit to my body and my day:** listening history and time of day alone ignore how the person is doing physically.
- **N4. Control of my data:** heart rate and sleep are sensitive; the user must know what is read and be able to delete it.

C. Existing Solutions

Table III compares the closest products we found, from their public pages; we did not install or test them.

TABLE III
COMPARISON WITH RELATED SOFTWARE

Solution	Body signals?	User's own songs?	Notes
Endel [3]	Yes: heart rate, time, weather	No: generates its own sounds	Paid; iOS, Android, Watch
Spotify daylist [4]	No: habits and time of day	Yes, through history	Updated several times a day
Spotify Prompted Playlists [5]	No: typed prompt and history	Yes, through history	Premium, US and Canada, beta
Our app	**Yes: heart rate, steps, sleep, time**	**Yes: saved library first**	**iPhone and Spotify first**

Endel reacts to the body but creates its own audio, and its claims about effects are company claims [3]. **Spotify** personalises from history and context, but the articles on daylist and Prompted Playlists mention no wearable [4], [5]. A systematic review of context-aware music recommenders (100 papers) finds that body signals are little studied, partly because suitable datasets are lacking [6]. We found no product that combines body signals and the user's own library; this rests on a limited search, so our argument is the added value of body signals and of a clear goal, not better accuracy than Spotify.

D. Proposed Solution

Fig. 1 shows the app in three parts. The person can always choose the situation with one tap and skip any song.

Song scores. Spotify removed its audio features (valence, energy) for new apps in November 2024 [7], so the app cannot analyse the sound. We use **MuSe**, a public dataset of 90,408 songs with valence and arousal scores built from Last.fm tags, with tentative Spotify IDs for about 68% of them [8]. For songs not in MuSe, a large language model (LLM) estimates the two scores from the title and artist; this estimate is checked on songs present in MuSe.

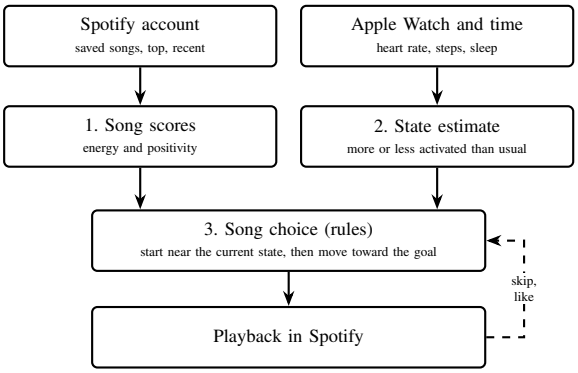


Fig. 1. System overview. The dashed arrow is the feedback from the listener.

State estimate. A small model says whether the person is currently more or less activated than usual, never a named emotion. Asking each user for weeks of personal data would make a poor first day, so we train on **EmoPhone** [9]: three yearly waves (2020–2022) of university students in Korea who wore a Fitbit and answered short mood questions about seven times a day for four weeks (297 participants after quality control). Its *arousal* answer, turned into "above or below this person's own average", is exactly our output. The model uses four inputs: the time of day, the heart rate, the steps and last night's sleep, each compared with the person's own average over the last 7 to 14 days, which needs no training on the user. A night or a period when the watch was not worn is marked as unknown, never as zero sleep or zero steps. The dataset authors granted us access for this project; the data files stay on team computers (NFR9).

Song choice. Rules build a queue of 3 to 5 songs: the first one or two match the current state, the next ones move step by step toward the goal of the situation. This follows the **iso principle** of music therapy, supported by a controlled experiment on mood after a sad film clip [10]. If the library has too few suitable songs, at most two MuSe songs close to the user's taste complete the queue and are marked as new.

An example. At 11:30 p.m. the user taps Wind-down. The heart rate is above the person's usual evening value, so the first two songs have a medium energy and the next ones become calmer step by step. A skip is recorded for future queues.

How we will test it. (1) Song scores: correlation between the LLM estimate and MuSe on shared songs. (2) State estimate: tests on people the model has not seen, never a random split of one person's data, then on a whole held-out wave, compared with a time-of-day baseline; if the model does not beat the baseline, it is not used. (3) Pilot: at most five Spotify accounts (the limit of Spotify's development mode [11]) for about two weeks, comparing skip rates and a one-tap "did this fit?" answer between our queues and queues built from the time of day only. The pilot measures fit, not health benefits.

Limits. The app is a demonstrator for five users and a wellness companion, not a medical tool. The training data come from a Fitbit and Android phones, not from an Apple Watch, so the pilot measures the remaining gap. Accuracy is

modest (Section V-D); the one-tap situation choice keeps the app useful when the estimate is wrong.

Going further. All music access goes through one small interface and all health data through another, so supporting Apple Music, Deezer or a watch on Android means writing one new connector (Section V-G).

E. Development Plan (revised)

Each milestone ends with a demo to the customer. The plan of Submission A put M1 at the end of October; the dates below replace it.

- **M1, Nov. 15:** Spotify sign-in and import tested with a live account, reading of Apple Health data, demo mode.
- **M2, Nov. 29:** song scores; state model trained on EmoPhone and tested on held-out people and a held-out wave.
- **M3, Dec. 6:** queue building, playback, feedback, explanation and data-control screens.
- **M4, Dec. 7 to end of term:** pilot (about two weeks), evaluation, final report and demo.

All code goes through pull requests on GitHub, reviewed by another member; the team meets weekly and decides together.

Changes since Submission A. EmoPhone replaces UC Irvine and PMData as training data (larger, in-the-moment labels, Korean students). The model inputs are now the time of day, the heart rate, the steps and last night's sleep. Heart-rate variability is no longer a model input, because EmoPhone does not contain it; it is still read for the explanation. EmoPhone has no watch-measured sleep either, so for training we estimate it from the Fitbit heart rate and steps (Section V-D). The weather is removed: EmoPhone locations are anonymised, so weather cannot be matched to each participant. The plan has new dates, and NFR9 is new.

III. Requirements

Priorities: Must, Should, Could. Requirements keep their numbers from Submission A and are grouped by module; "(rev.)" marks a change.

A. Spotify Connection

The app must connect to the user's Spotify account, read the library and play music.

- **FR1. Sign-in (Must):** connect through Spotify's secure sign-in (OAuth) and disconnect at any time.
- **FR2. Library import (Must):** download all saved songs (title, artists, cover art), page by page, and store them on the phone.
- **FR9. Playback (Must):** start the queue in Spotify; if this fails, open the songs in the Spotify app.
- **NFR6. Spotify rules:** show cover art unaltered, the Spotify logo and a link to Spotify.

B. Watch Data and Personal Average

- **FR3. Watch data (Must, rev.):** with permission, read heart rate, heart-rate variability, sleep and steps from Apple Health when the app is opened. Heart rate, steps and sleep feed the model; heart-rate variability feeds the explanation.
- **FR4. Personal average (Must):** compute the person's usual values over 7 to 14 days and compare recent values with them.
- **NFR4. Missing data and watch not worn:** a night or a period without heart-rate samples is marked as unknown, never as zero sleep or zero steps; if watch data are missing or old, the app still works from the time of day and the library.

C. Song Scores

- **FR6. Song scores (Must):** give each song an energy and a positivity score from MuSe, or from an LLM estimate when it is not found (Should).

D. State Estimate

- **FR7. State estimate (Must, rev.):** estimate whether the person is more or less activated than usual with a model trained on EmoPhone.
- **NFR7. Responsible use:** no medical claim, no emotion label shown to the user; the user can always stop or change the music.

E. Situation and Queue

- **FR5. Situation choice (Must):** four one-tap buttons (Table II), with one suggested from the time, the state and last night's sleep (Should).
- **FR8. Queue building (Must):** 3 to 5 songs from the library, starting near the current state and moving toward the goal, with at most two matching songs; at most two MuSe songs when the library is not enough (Should).
- **FR11. Explanation (Should):** a short reason, e.g. "short night, morning: calm songs first, then more energy".
- **NFR3. Speed:** a queue within 2 s of the tap; 2,000 songs imported in under 60 s on Wi-Fi.
- **NFR5. Ease of use:** at most two taps from launch to music.

F. Feedback, Data Control and Security

- **FR10. Feedback (Must):** record skips, likes and a "did this fit?" answer, and avoid disliked songs.
- **FR13. Data control (Must):** view, export and delete all stored data.
- **NFR1. Privacy:** health data stay on the phone; the LLM only receives titles and artists; explicit consent per data source.
- **NFR2. Security:** Spotify tokens in the iOS Keychain; no API key inside the app; HTTPS only.

G. Demo Mode, Tests and Other Services

- **FR12. Demo mode (Should):** run the app on recorded data, without a live watch.
- **NFR8. Testability:** song scores, state estimate and song choice covered by automated tests on recorded data.
- **FR14. Other services (Could):** support another music service or watch through the same interfaces.

H. Dataset Terms

- **NFR9. Dataset terms (new):** EmoPhone data are used on team computers only, never committed to the public repository or shipped in the app.

IV. Development Environment

A. Choice of Software Development Platform

The app is developed as a native **iOS application**. The Apple Watch data are only available on the iPhone through **HealthKit**, and the **Spotify iOS SDK**, which controls playback, also runs on the iPhone; a web app could read neither. Building an iPhone app requires **Xcode** on **macOS**, so the app is developed on a MacBook. The data and AI part is written in **Python** and runs on a Windows computer; only the final conversion to Core ML is done on the Mac. The trained model is converted to **Core ML** and runs on the phone, so no database server is needed: health data never leave the phone (NFR1). One small cloud part remains: the LLM API key must not be stored in the app, because anyone could extract it from the installed app. Song-score requests therefore go through a **Cloudflare Worker**, a serverless function that holds the key, accepts only titles and artists, and limits the number of requests per user. The free plan allows 100,000 requests per day [18], far above our needs. Table IV explains each choice.

TABLE IV
Tools and Language Choice

Tool / language	Reason
Swift + SwiftUI	Apple's main language; HealthKit and Core ML are designed for it; SwiftUI builds screens with little code.
HealthKit	Only way to read Apple Watch heart rate, heart-rate variability, sleep and steps, with per-signal consent.
Spotify Web API + iOS SDK	Web API for the library and track lookup; iOS SDK to play and queue songs in the Spotify app.
SwiftData	Local storage on the phone for songs, scores, summaries and feedback, with no server.
Python, pandas, scikit-learn	Reading the dataset files; logistic regression and `GroupKFold` to test on people kept apart.
coremltools + Core ML	Converts the trained model so it runs on the phone, offline.
LLM API (Claude Haiku 5.5)	Scores songs missing from MuSe from title and artist, in batches, cached; low cost (final model chosen in the design phase).
Cloudflare Workers	Serverless proxy that keeps the LLM API key out of the app and limits requests; free plan, no server to maintain.
GitHub + LaTeX	Version control and pull-request reviews; documents in the IEEE template.

Development environment. Table V gives the machines and versions. scikit-learn is kept at version 1.5, the latest supported by coremltools 8 for model conversion [16].

TABLE V
Development Environment

Item	Version or resources
Mac (app)	MacBook Air M3, 24 GB RAM; macOS 27.2
Xcode, Swift	Xcode 27.2, Swift 6 language mode
App target	iOS 18.0 minimum (SwiftData, current HealthKit APIs)
iPhone, Apple Watch	iPhone 16 (iOS 27.2); Apple Watch Series 9 (watchOS 27.2)
Computer (data and AI)	Windows 11, 32 GB RAM
Python and libraries	Python 3.12; pandas 2.2, scikit-learn 1.5, coremltools 8.3 (model conversion run on the Mac)
Spotify iOS SDK	5.0.1, through Swift Package Manager
Cloud	Cloudflare Workers (free plan), deployed with the Wrangler command-line tool

B. Cost Estimation

The team uses Macs, iPhones and watches it already owns, and all development tools are free (Xcode, Python, GitHub, LaTeX). Four costs remain (Table VI). The **Apple Developer Program** costs US$99 per year [13]; one membership is enough for the team, and without it an app installed from Xcode expires after 7 days [14]. **Spotify Premium** is required for the app owner in development mode and for playback [11]; members already subscribed pay nothing extra [15]. The **LLM** cost is negligible: about 6,000 songs (four libraries plus the check on MuSe) at about 200 tokens each, at the batch price of Claude Haiku 5.5 [19], is under US$1. The team also uses AI assistants to help with development: four **Claude Pro** subscriptions at US$20 per month [17] for the two months of the project, US$160 in total. The Cloudflare Worker stays within the free plan [18].

TABLE VI
Cost Estimation (prices checked Nov. 4, 2026)

Item	Cost
Apple Developer Program (one for the team)	US$99 per year
Spotify Premium, per account not yet subscribed	KRW 11,990 / month (KRW 6,600 student)
LLM song scores	under US$1 in total
Claude Pro for development (4 members, 2 months)	$4 \times 2 \times$ US$20 = US$160
Cloudflare Workers (LLM proxy)	free plan
Hardware and software tools	already owned or free

C. Software in Use

1) Existing Systems / Tools: **Endel** [3] adapts generated soundscapes to heart rate, time and weather. It shows that body signals are useful for music, but it does not play the user's own songs.

Spotify daylist and Prompted Playlists [4], [5] personalise playlists from listening history, time of day or a typed prompt. They play the user's music but ignore the body.

MuSe [8] gives valence and arousal for 90,408 songs (CC BY 4.0). We use it directly as song scores.

EmoPhone and its benchmark [9] provide the training data and an evaluation method: features from the minutes before each mood answer, normalisation per person, and labels relative to each person's average. On people not seen in training, its best models reach an AUROC of about 0.58 for

arousal with thousands of phone and watch features (0.5 is chance).

WatchAnxiety [12] predicts a yes/no anxiety state from smartwatch heart rate and reports 60.4% balanced accuracy, another sign that realistic results are modest.

2) Comparison & Gap Analysis:

- **Body and own music:** Endel uses the body but not the user's songs; Spotify uses the user's songs but not the body. Our app combines both.
- **Clear goal:** existing playlists follow the user's habits; our queue has a goal (calmer or more energetic) and a path toward it.
- **Transparency:** simple rules and a small model, so each queue can be explained to the user (FR11) and tested.
- **Privacy:** the model runs on the phone; health data are never sent to a server, only song titles to the LLM proxy.

D. Task Distribution

The detailed task distribution will be given in the design phase, once the modules and their interfaces are fixed. Until then, the work follows the roles of Table I.

V. Specifications

Each requirement group of Section III is described with its goal, implementation and pseudo-code. This section is a plan: the code is written in the next phases.

A. Requirement 1 – Spotify Connection (FR1, FR2, FR9, NFR6)

Goal: Connect the user's Spotify account, import the library and play a queue.

Implementation:

- **App:** "Connect Spotify" screen; OAuth with PKCE, so no client secret is stored in the app; tokens in the Keychain (NFR2).
- **Spotify Web API:** the February 2026 changes keep the endpoints we need: `GET /me/tracks`, `/me/top/tracks`, `/me/player/recently-played` and `GET /tracks/{id}`. The batch lookup `GET /tracks` was removed, and saving a song now uses `PUT /me/library` [11]. Each call is checked with a live account in M1.
- **Playback:** Spotify iOS SDK plays the first song and adds the others to the Spotify queue; if the SDK cannot connect, the song opens in the Spotify app.
- **Screen:** cover art unaltered, Spotify logo and link (NFR6).

Pseudocode:

```
connectSpotify():                  # OAuth with PKCE
  code   = openSpotifyLogin(sha256(verifier))
  tokens = exchange(code, verifier) -> Keychain
importLibrary():
  url = "/me/tracks?limit=50"
  while url:
    page = GET(url)                  # retry on 429
    db.tracks.upsert(page.items); url = page.next
  add top tracks and recent plays; score new songs
```

B. Requirement 2 – Watch Data and Personal Average (FR3, FR4, NFR4)

Goal: Read the watch data and compare recent values with the person's own usual values.

Implementation:

- **HealthKit:** anchored queries on opening (Apple Health cannot be read while the phone is locked); one consent per signal.
- **Summaries:** raw samples are turned into hourly and nightly summaries, then dropped (NFR1).
- **Personal average:** mean and standard deviation per 3-hour block of the day over the last 14 days, ready after 7 days; heart rate depends strongly on the time of day.
- **Sleep:** HealthKit sleep samples recorded by the Apple Watch only; "in bed" periods set on the iPhone are ignored.
- **Watch worn?** The watch measures heart rate only when it is on the wrist, so a night counts only if heart-rate samples cover at least 60% of the 21:00–12:00 window; otherwise last night's sleep is *unknown*, not zero. The same rule applies to steps in the last 30 minutes (NFR4).
- **Missing data:** an unknown value is replaced by the person's usual value and flagged for the model; without heart rate in the last 30 minutes, the app falls back on the time of day.

Pseudocode:

```
onAppOpen():
  for s in [heartRate, hrv, sleep, steps] if consent(s):
    samples = HealthKit.query(s, after=anchor[s])
    db.summaries.update(hourly(samples))
  night = last 21:00-12:00 window
  worn  = share of minutes with heart rate in night
  sleep = watchSleep(night) if worn >= 0.6 else UNKNOWN
  for s, block in signals x 3-hour blocks:
    v = summaries(s, block, last 14 days)
    baseline[s, block] = (mean(v), sd(v)) if days(v) >= 7
                         else NONE
```

C. Requirement 3 – Song Scores (FR6)

Goal: Give every library song an energy and a positivity score between 0 and 1.

Implementation:

- **MuSe table:** shipped with the app (title, artists, scores, Spotify ID), rescaled to 0–1; matched by Spotify ID, then by normalised title and artist.
- **LLM:** songs still missing are sent in batches of 50 (titles and artists only) to the Cloudflare Worker, which adds the API key and calls the LLM; the JSON answer is checked and cached, so each song is sent once.
- **Check:** offline, the same prompt is run on 2,000 MuSe songs; we report the correlation and the mean error.

Pseudocode:

```
for t in tracks without score:
  m = muse.byId(t.id) or muse.byTitleArtist(norm(t))
  if m: save(t, m.arousal, m.valence, origin="MuSe")
  else: missing.append(t)
for batch in chunks(missing, 50):
  scores = POST worker/score {titlesAndArtists(batch)}
  save(batch, valid(scores) or 0.5, origin="LLM")
```

D. Requirement 4 – State Estimate (FR7, NFR7)

Goal: Estimate, between 0 and 1, whether the person is more or less activated than usual.

Implementation:

- **Training data:** EmoPhone. For each mood answer: the heart rate and the steps in the 15, 30 and 60 minutes before it, last night's sleep duration and the time since waking up, all compared with the person's own average, plus the hour of the day. The label is 1 when the arousal answer is above the person's own average.
- **Sleep in training:** EmoPhone has no watch-measured sleep, so we estimate it the way watches do: the longest period between 21:00 and 12:00 with the watch worn, no steps and a heart rate in the person's lowest 30%, lasting at least 2 hours. The median is 7.3 hours. The watch was not worn on 34% of nights; those nights are marked unknown, as in the app.
- **Model:** logistic regression (scikit-learn), tested with a 5-fold split by participant; kept only if it beats the time-of-day baseline; converted with coremltools.
- **On the phone:** the same features come from Requirement 2; Core ML returns a number, never an emotion label (NFR7).

Pseudocode:

```
# offline, Python
x = [z(HR_15), z(HR_30), z(HR_60), z_block(HR_30),
     z(steps_15), z(steps_30), z(steps_60),
     z(sleep), hoursSinceWake, sleepUnknown,
     sin(hour), cos(hour), weekend]
y = arousal > mean arousal of this participant
for fold in GroupKFold(5, groups=participant):
  fit LogisticRegression; report AUROC, balanced accuracy
keep if AUROC > time-only baseline -> StateModel.mlmodel

# on the phone
if no baseline or no heart rate in last 30 min:
  return prior(hour)              # time-of-day mode
# 0 = calmer than usual .. 1 = more activated
return StateModel.predict(x)
```

First results. We ran this pipeline on the first EmoPhone wave (92 participants, 10,259 answers); Table VII gives the results. Our four-input model reaches an AUROC of 0.573, the level of the best published models on this wave, which use thousands of phone and watch features [9]. Heart rate carries most of the information. On this wave, steps and sleep do not add a measurable gain: the four-input model is within the variation between folds ($\pm$0.015) of the heart-rate model. We keep them because the Apple Watch measures sleep itself, whereas EmoPhone sleep had to be estimated, and because they drive the rules. The held-out waves and the pilot will show whether they help; if not, the app uses the heart-rate model. The result is unchanged with one heart-rate reading every 5 minutes, which matters because the Apple Watch is expected to measure less often than the Fitbit. Stress is barely predictable, so the Stress situation is chosen by the user. The mood of the *other* participants at the same date and time, which includes shared effects such as the weather, gives only 0.53, so removing the weather costs little. These are first results on one wave.

TABLE VII
FIRST RESULTS ON EMOPHONE WAVE 1 (PEOPLE KEPT APART)

Target and inputs	AUROC	Bal. acc.
Arousal: time of day only	0.532	0.509
Arousal: time + sleep	0.526	0.507
Arousal: time + heart rate	0.577	0.547
Arousal: time + heart rate + steps	0.577	0.547
Arousal: time + heart rate + steps + sleep (our model)	**0.573**	**0.542**
Arousal: same, gradient-boosted trees	0.558	0.533
Arousal: time + heart rate, one reading every 5 min	0.575	–
Stress: our model	0.543	0.514
Arousal: other participants at same date and time	0.530	–

E. Requirement 5 – Situation and Queue (FR5, FR8, FR11, NFR3, NFR5)

Goal: Suggest a situation and build a short queue that goes from the current state to the goal.

Implementation:

- **Home screen:** four large buttons; the suggested one is highlighted: wake-up within two hours after the end of sleep, wind-down after 21:00, low energy between 13:00 and 17:00 when the state is below 0.4.
- **Queue rules:** target energies go from the state to the goal of the situation (e.g. 0.2 for wind-down, 0.7 for wake-up); for each position, the closest library song is chosen, preferring positive songs for stress and low energy, avoiding repeated artists and songs played in the last 3 days.
- **MuSe fallback:** when no library song is within 0.15 of the target, a MuSe song with a shared artist or genre is checked with `GET /tracks/{id}` and marked as new (at most two).
- **Explanation:** built from fixed sentence parts, e.g. "heart rate higher than usual: medium songs first, then calmer".
- **Speed:** the queue is built from local data, with at most a few lookups (NFR3).

Pseudocode:

```
buildQueue(situation, e0):     # e0 = state estimate
  g = goal[situation]
  n = 5 if |e0 - g| > 0.3 else 3
  m = 2 if n == 5 else 1       # matching songs
  targets = [e0]*m + evenly(e0 -> g, n - m)
  for e in targets:
    c = closest library song to e
        (not recent, not skipped, with penalties)
    if |energy(c) - e| > 0.15 and suggestions < 2:
      c = museSuggestion(e)    # checked on Spotify
    queue.add(c)
  return queue, explain(context, plan)
```

F. Requirement 6 – Feedback, Data Control and Security (FR10, FR13, NFR1, NFR2)

Goal: Learn from skips and likes, and give the user full control over the stored data.

Implementation:

- **Player screen:** skip, like and, after the queue, "did this fit?". A song skipped twice in a situation is left out of it; a skipped suggestion is never proposed again; a like saves the song to the library.

- **Settings screen:** lists the stored data and consents; export as one JSON file; delete everything after a confirmation.
- **Security:** tokens in the Keychain, HTTPS only (App Transport Security on); the LLM key stays in the Cloudflare Worker, and the request contains titles and artists only.

Pseudocode:

```
on skip (before 30 s): save(skip)
on like:               save(like); PUT /me/library
after last song:       ask "Did this fit?"
deleteAll():
  clear tracks, scores, summaries, queues,
        feedback, consents
  Keychain.delete(tokens); reset HealthKit anchors
```

G. Requirement 7 – Demo Mode, Tests and Other Services (FR12, FR14, NFR8)

Goal: Run and test the app without a live watch, and keep Spotify and Apple Health replaceable.

Implementation:

- **Interfaces:** the song scores, state estimate and queue rules only see two Swift protocols. A new service (Apple Music, Deezer, Health Connect on Android) means one new implementation.
- **Demo mode:** both protocols implemented with recorded files (a few days of a team member's summaries, a small song list); no EmoPhone data (NFR9).
- **Tests:** XCTest unit tests on these files (z-scores, baselines, queue rules such as "never more than two suggestions", LLM answer parser); pytest for the Python code.

```
protocol MusicProvider {
  func signIn() async throws
  func importLibrary() async throws -> [Track]
  func play(_ q: Queue) async throws
  func save(_ t: Track) async throws
}
protocol HealthProvider {
  func requestConsent(_ s: Set<Signal>) async
  func newSamples(_ s: Signal) async -> [Sample]
}
```

H. Requirement 8 – Data Storage and Dataset Terms (NFR1, NFR9)

Goal: Store all app data on the phone, and keep the training data out of the public repository.

Implementation:

- **Phone:** SwiftData tables below; the MuSe table and the Core ML model are read-only files in the app; tokens in the Keychain.
- **Repository:** `data/` is listed in `.gitignore`, and the repository is checked before each submission (NFR9).

Schema:

```
tracks:    { spotifyId, title, artists[], coverURL,
             source, addedAt }
scores:    { spotifyId, energy, positivity, origin }
hours:     { hourStart, meanHR, sdnnHRV, steps }
nights:    { date, sleepMinutes, sleepEnd, worn }
baselines: { signal, block3h, mean, sd, nDays }
queues:    { id, situation, startEnergy, kind,
             reason, items[] }
feedback:  { queueId, spotifyId, event, seconds, time }
consents:  { source, granted, time }
Keychain:  { accessToken, refreshToken, expiresAt }
```

I. System Integration Workflow

Data flow:

```
[Apple Health] -> Health connector -> hours, baselines
                                            |
[Spotify] -> Music connector -> tracks -> Song scores
                     (MuSe / LLM via Cloudflare Worker)
                                            |
user taps a situation -> State model (Core ML)
                                            |
                     Queue rules + explanation
                                            |
[Spotify app] <- play / queue     skip, like -> feedback
```

REFERENCES

[1] Y. Whan-woo, "Sleep market in Korea reaches W3 tril. says report," *The Korea Times*, Apr. 10, 2024. [Online]. Available: https://www.koreatimes.co.kr/www/biz/2024/04/126_372435.html

[2] K. V. Jespersen, V. Pando-Naude, J. Koenig, P. Jennum, and P. Vuust, "Listening to music for insomnia in adults," *Cochrane Database Syst. Rev.*, no. 8, Art. no. CD010459, 2022, doi: 10.1002/14651858.CD010459.pub3.

[3] Endel, "Endel: personalized soundscapes." [Online]. Available: https://endel.io/ (accessed Oct. 20, 2026).

[4] "Spotify launches ever-changing 'Daylist' playlist that adapts to users' moods," *Music Business Worldwide*, 2023. [Online]. Available: https://musicbusinessworldwide.com/spotify-launches-ever-changing-daylist-playlist-that-adapts-to-users-moods

[5] "Spotify brings AI-powered Prompted Playlists to the U.S. and Canada," *TechCrunch*, Jan. 22, 2026. [Online]. Available: https://techcrunch.com/2026/01/22/spotify-brings-ai-powered-prompted-playlists-to-the-u-s-and-canada/

[6] Á. Lozano Murciego, D. M. Jiménez-Bravo, A. Valera Román, J. F. De Paz Santana, and M. N. Moreno-García, "Context-aware recommender systems in the music domain: a systematic literature review," *Electronics*, vol. 10, no. 13, Art. no. 1555, 2021.

[7] Spotify for Developers, "Introducing some changes to our Web API," Nov. 27, 2024. [Online]. Available: https://developer.spotify.com/blog/2024-11-27-changes-to-the-web-api

[8] C. Akiki and M. Burghardt, "MuSe: The Musical Sentiment Dataset," *J. Open Humanit. Data*, vol. 7, 2021, doi: 10.5334/johd.33.

[9] P. Zhang, M. Park, S. Kang *et al.*, "EmoPhone: A multi-wave dataset for in-the-wild mobile and wearable affect sensing," arXiv:2609.16581, 2026. Data: Harvard Dataverse, doi: 10.7910/DVN/QGRVU8.

[10] K. Starcke, J. Mayr, and R. von Georgi, "Emotion modulation through music after sadness induction—the iso principle in a controlled experimental study," *Int. J. Environ. Res. Public Health*, vol. 18, no. 23, Art. no. 12486, 2021.

[11] Spotify for Developers, "Web API changes, February 2026" and "February 2026 dev mode changes: migration guide." [Online]. Available: https://developer.spotify.com/documentation/web-api/references/changes/february-2026 (accessed Nov. 4, 2026).

[12] M. S. Ahmed *et al.*, "WatchAnxiety: a transfer learning approach for state anxiety prediction from smartwatch data," arXiv:2509.13725, 2025.

[13] Apple, "Apple Developer Program." [Online]. Available: https://developer.apple.com/programs/ (accessed Nov. 4, 2026).

[14] Apple, "Choosing a membership." [Online]. Available: https://developer.apple.com/support/compare-memberships (accessed Nov. 4, 2026).

[15] Spotify, "Premium plans (Korea)." [Online]. Available: https://www.spotify.com/kr-en/premium (accessed Nov. 4, 2026).

[16] Apple, "coremltools releases." [Online]. Available: https://github.com/apple/coremltools/releases (accessed Nov. 4, 2026).

[17] Anthropic, "Plans & pricing." [Online]. Available: https://claude.com/pricing (accessed Nov. 4, 2026).

[18] Cloudflare, "Limits," Cloudflare Workers docs. [Online]. Available: https://developers.cloudflare.com/workers/platform/limits/ (accessed Nov. 4, 2026).

[19] Anthropic, "Pricing," Claude Platform Docs. [Online]. Available: https://platform.claude.com/docs/en/about-claude/pricing (accessed Nov. 4, 2026).